Stabilizing & Accelerating Mahalanobis MMD

Reboosting the power of two sample test

Ruijie Li, Ziheng Zheng, Dechao Huang, Yao yu

June 2026

Two-Sample Testing

The question, and what a test must provide

We observe two samples and ask whether they come from the *same* distribution:

$$\mathcal{X}_m = \{X_1, \ldots, X_m\} \sim P, \qquad \mathcal{Y}_n = \{Y_1, \ldots, Y_n\} \sim Q, \qquad H_0 : \; P = Q \;\text{ vs }\; H_1 : \; P \neq Q.$$

A test needs two ingredients:

- a **statistic** that measures how different the two samples look;
- a **decision rule** – reject H_0 when that statistic is large.

The whole talk is about building a good statistic and keeping it reliable.

Maximum Mean Discrepancy (MMD)

The single-kernel building block

MMD measures the distance between two distributions through a **kernel** K (a similarity function). Its key property: $\mathrm{MMD} = 0 \iff P = Q$.

From data we estimate it by averaging *within*-sample similarities minus *between*-sample similarities:

$$\mathrm{MMD}^2[\mathsf{K}, \mathcal{X}_m, \mathcal{Y}_n] = \underbrace{\mathcal{W}_{\mathcal{X}_m} + \mathcal{W}_{\mathcal{Y}_n}}_{\text{within}} - 2\,\underbrace{\mathcal{B}_{\mathcal{X}_m,\mathcal{Y}_n}}_{\text{between}}.$$

The single-kernel test rejects H_0 when this estimate is large.

Weakness: for a Gaussian kernel $\mathsf{K}_\sigma(x, y) = e^{-\|x-y\|^2/\sigma^2}$, the bandwidth σ fixes the scale – small σ sees local, large σ sees global differences. One σ is a gamble.

The Paper's Method: Mahalanobis MMD

Combine many bandwidths into one statistic

Instead of betting on one bandwidth, use r kernels $\mathrm{K}_1, \ldots, \mathrm{K}_r$ and collect their MMD estimates into a vector $\mathrm{MMD}^2[\mathcal{K}, \mathcal{X}_m, \mathcal{Y}_n]$. Simply summing them would *double-count* correlated kernels, so we combine them by their Mahalanobis distance – weighting by the inverse covariance $\hat{\mathbf{\Sigma}}^{-1}$:

$$\boxed{T_{m,n} = \left(\mathrm{MMD}^2[\mathcal{K}, \mathcal{X}_m, \mathcal{Y}_n]\right)^\top \hat{\mathbf{\Sigma}}^{-1} \left(\mathrm{MMD}^2[\mathcal{K}, \mathcal{X}_m, \mathcal{Y}_n]\right)}$$

Decision rule. Reject H_0 when $T_{m,n}$ exceeds a cut-off; the cut-off is calibrated by a multiplier bootstrap. A large $T_{m,n}$ means the kernels *jointly* detect a difference.

MMMD Framework

Mahalanobis Multi-Kernel Maximum Mean Discrepancy

A multi-kernel two-sample test with decorrelated kernel aggregation and multiplier bootstrap.

Samples $X \sim P,\ Y \sim Q$ → Multi-kernel $\{k_1, \ldots, k_r\}$ → MMD vector $\hat{\eta} \in \mathbb{R}^r$ → Mahalanobis decorrelation → Multiplier bootstrap

Kernel families

- GEXP: RBF exponential bandwidth grid
- LAP: Laplace kernel family
- MIXED: combined RBF + Laplace

Key components

- $\hat{\Sigma}$: kernel covariance estimation
- Ridge stabilization: $\lambda = 10^{-5} \cdot \min(\mathrm{diag}(\hat{\Sigma}))$
- Vectorised bootstrap for α-grid scan

Extensions

- Graph-MMMD: kNN graph + heat kernel
- Skew-normal, MV-Laplace, Normal-t mixture
- Parallel computing backend

The Covariance $\hat{\Sigma}$

The engine behind the Mahalanobis weighting

The weighting needs $\hat{\Sigma}$: the covariance of the r MMD estimates under H_0. It is what puts correlated kernels on a common footing. Its empirical estimate (paper, Eq. 15) is

$$\hat{\sigma}_{ab} = \frac{2}{\hat{\rho}^2(1-\hat{\rho})^2} \cdot \frac{1}{m^2} \sum_{1 \leq i,j \leq m} \hat{\mathsf{K}}_a^{\circ}(X_i, X_j)\, \hat{\mathsf{K}}_b^{\circ}(X_i, X_j), \qquad \hat{\rho} = \tfrac{m}{m+n}.$$

Every entry is built from **kernel Gram matrices** on the X sample – and that is exactly where both of my fixes act.

Gram Matrices and a Hidden Geometry

Why $\hat{\mathbf{\Sigma}}$ is fragile and costly

The **Gram matrix** of a kernel collects all pairwise similarities, $\hat{\mathsf{K}}_a = \{\mathsf{K}_a(X_i, X_j)\}$. After centering, $\hat{\mathbf{\Sigma}}$ turns out to be a "Gram matrix of Gram matrices" – each entry is an inner product of two centered Gram matrices:

$$\hat{\sigma}_{ab} \;\propto\; \left\langle \hat{\mathbf{K}}_a^\circ, \hat{\mathbf{K}}_b^\circ \right\rangle_F, \qquad \hat{\mathbf{K}}_a^\circ = \tfrac{1}{m} C_m \hat{\mathsf{K}}_a C_m \qquad \Longrightarrow \qquad \hat{\mathbf{\Sigma}} = \text{Gram matrix of } \{\mathrm{vec}(\hat{\mathbf{K}}_a^\circ)\}.$$

Ill-conditioned. Redundant bandwidths give near-identical Gram matrices, so $\hat{\mathbf{\Sigma}}$ is near-singular; then $\hat{\mathbf{\Sigma}}^{-1}$ blows up estimation noise and H_0 is over-rejected.

Expensive. A naive build needs R^2 such inner products.

My Extension: Two Construction-Layer Fixes

Attack the cause, not the symptom

- **Stability.** $\hat{\Sigma}$ is ill-conditioned. Fix: prune the redundant kernels *before* inverting – lazy pivoted Cholesky.

- **Speed.** The naive $O(R^2)$ build is slow. Fix: exploit the Gaussian/Laplace product structure – multiplicative closure.

Both act while $\hat{\Sigma}$ is being *built*. The paper's only safeguard is a fixed ridge $(\hat{\Sigma} + \lambda I)^{-1}$ added at the *solve* step, which masks the problem rather than removing it.

Fix 1 – Stability: Lazy Pivoted Cholesky

Keep only kernels that add something new

Idea. Go greedily: each round keep the kernel whose Gram matrix is *least explained* by the ones already chosen, and stop once the rest add almost nothing. Redundant kernels are dropped, so $R \to q \ll R$ and $\hat{\boldsymbol{\Sigma}}$ is well-conditioned again. "*Lazy*": a covariance column is computed only when its kernel is picked, so the full $R \times R$ matrix is never formed.

Lazy Pivoted-Cholesky Kernel Selection

1. residuals $r_j \leftarrow \hat{\boldsymbol{\Sigma}}_{jj}$; pivot $p_t = \arg\max_j r_j$; **stop** if $r_{p_t} \leq \varepsilon \max_j \hat{\boldsymbol{\Sigma}}_{jj}$.
2. fetch column $\mathcal{C}(p_t) = \hat{\boldsymbol{\Sigma}}_{\cdot, p_t}$ (lazy); $L_{j,t} = \left(\mathcal{C}(p_t)_j - \sum_{s<t} L_{j,s} L_{p_t,s}\right) / \sqrt{r_{p_t}}$.
3. update residuals $r_j \leftarrow r_j - L_{j,t}^2$; repeat.

Fix 2 – Speed: Multiplicative Closure

Reuse one shared function instead of R^2 scans

Idea. Gaussian/Laplace kernels multiply into another kernel of the same family. So a covariance entry depends only on the *sum* of the two bandwidth parameters, through a single shared function H:

$$\mathrm{K}_{t_a}\mathrm{K}_{t_b} = \mathrm{K}_{t_a+t_b} \quad (t = 1/\sigma^2), \qquad H(u) = \sum_{i,j} e^{-uD_{ij}}, \quad D_{ij} = \|X_i - X_j\|^2.$$

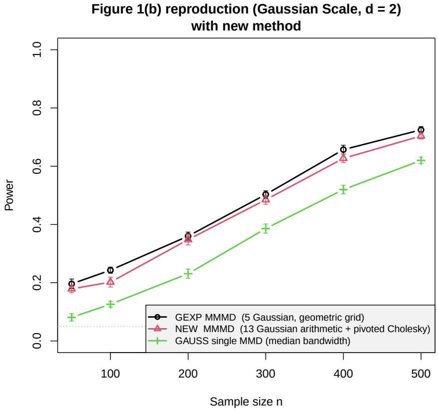
On an arithmetic grid, $t_a + t_b$ takes only $2R - 1$ distinct values, so we compute $O(R)$ values of H rather than R^2.

$$\text{Covariance cost:} \quad \underbrace{O(R^2 m^3 + BRm^2)}_{\text{naive}} \longrightarrow \underbrace{O(Rm^2 + Bqm^2)}_{\text{closure + selection}}$$

The heavy m^2 work is cached once; later steps (and each lazy column query) are cheap.

Experiments: Power

Synthetic Gaussian scale shift – matches the paper

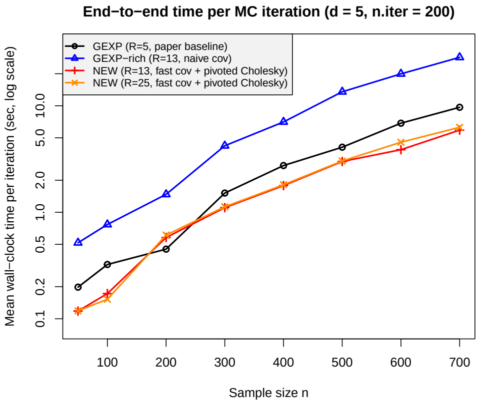


$d = 2$ variance shift;
x-axis: sample size n,
y-axis: power.

The new method tracks the paper's multi-kernel test to within 4 pp, and both clearly beat the single-kernel test.

Experiments: Speed

Large speed-up at equal kernel count

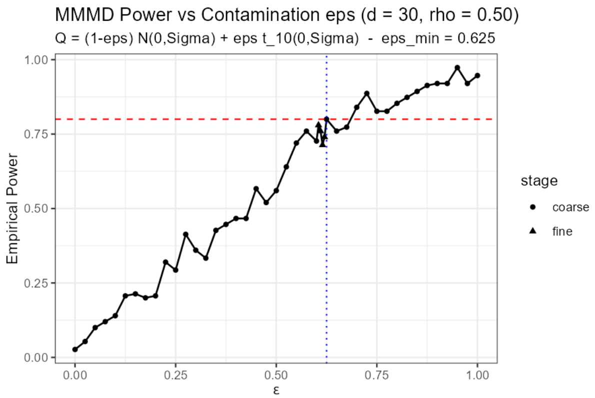


$d = 5$;
x-axis: sample size n,
y-axis: time per iteration.

At the same $R = 13$: covariance $\sim 16\times$ and end-to-end $\sim 4.8\times$ faster; cost grows *linearly* in R, not quadratically.

Task 1: ε-Contamination Sensitivity

How much t_{10} contamination can MMMD detect?



Setup

$P = \mathcal{N}(0, \Sigma)$,

$Q = (1 - \varepsilon)\mathcal{N}(0, \Sigma) + \varepsilon \cdot t_{10}(0, \Sigma)$

- $d = 30$, $n = 100$, $\rho = 0.5$ (AR(1))
- GEXP kernel, $r = 5$, $B = 500$, $N = 150$

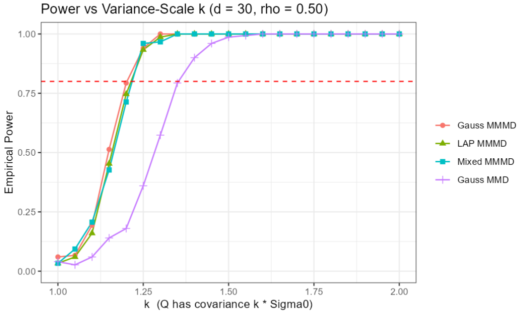
$\varepsilon_{\min} = 0.625$

Power reaches 0.80 when 62.5% of Q is t_{10}.

Steep transition: $\varepsilon \in [0.45, 0.65]$, power rises $0.57 \to 0.80$.

Task 2: Variance-Scale k Sensitivity

How small a variance change can MMMD detect?



Setup

$$P = \mathcal{N}(0, \Sigma_0), \qquad Q = \mathcal{N}(0, k \cdot \Sigma_0)$$

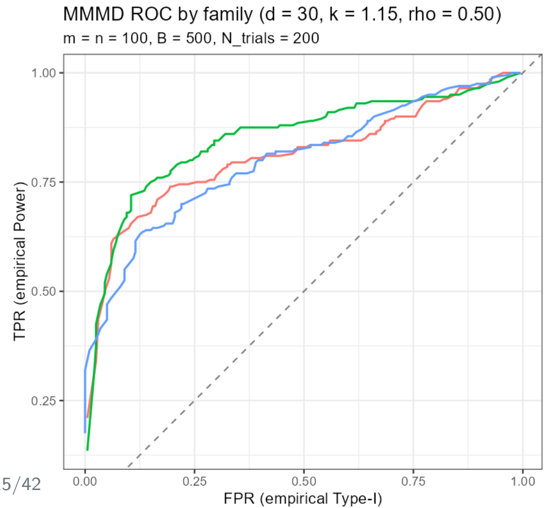
Method	$k_{\min}$	Power at $k = 1.20$
Mixed MMMD	**1.25**	0.71
Gauss MMMD	1.25	0.79
LAP MMMD	1.25	0.75
Gauss MMD (single)	1.40	0.18

MMMD: 15% earlier detection than single-kernel MMD

At $k = 1.20$, MMMD achieves 71–79% power while single-kernel MMD stays at 18% — a 4× gap.

Task 3: Type-I Error & ROC Analysis

Power–Type-I trade-off across kernel families



Type-I Error @ $\alpha = 0.05$

Family	$r = 3$	$r = 5$	$r = 10$
GEXP	0.020	0.055	0.120
LAP	0.045	0.045	0.050
MIXED	0.050	0.050	0.070

FPR & TPR (ROC, $k = 1.15$)

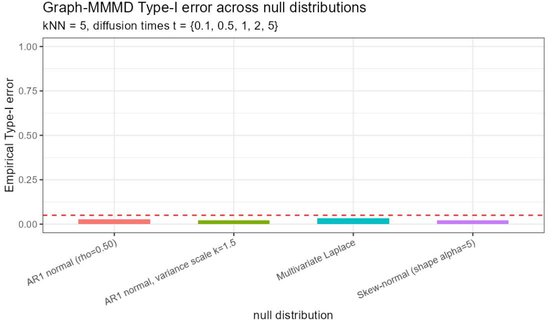
Family	FPR	TPR
GEXP	0.045	0.500
LAP	0.045	0.495
MIXED	0.050	0.435

- $r = 3, 5$: well-calibrated; $r = 10$ GEXP inflates (0.12) — too many kernels degrade $\hat{\Sigma}$

Task 4a: Graph-MMMD Extension

kNN graph kernel with heat/diffusion kernel

Graph-MMMD: build kNN graph → graph Laplacian → heat kernel $K_t = \exp(-tL)$ → MMMD pipeline.



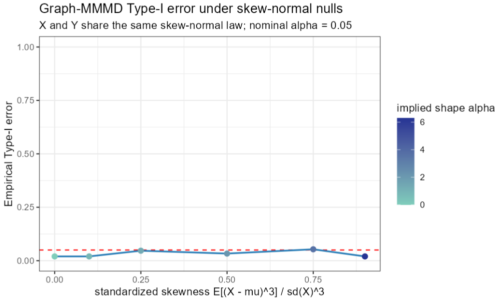
Type-I under 4 null distributions

Null Distribution	Type-I Error
AR(1) Normal ($\rho = 0.5$)	0.027
AR(1) Normal, $k = 1.5$	0.020
Skew-Normal ($\alpha = 5$)	0.020
Multivariate Laplace	0.033

- All ≤ 0.053 — well-calibrated across distributions
- Non-Gaussian and asymmetric nulls do not inflate Type-I

Task 4b: Skewness Robustness

Graph-MMMD calibration across skewness levels



Skewness sweep

Skew-Normal with shape α mapped to standardized skewness.

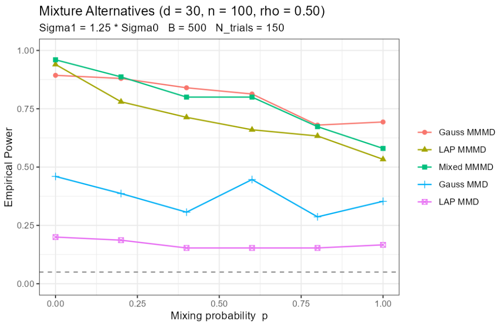
Std. Skewness	Shape α	Type-I
0.00	0.00	0.020
0.10	0.87	0.020
0.25	1.35	0.047
0.50	2.17	0.033
0.75	3.64	0.053
0.90	6.30	0.020

Type-I ≤ 0.053 even at extreme skewness (0.90) α

found via `uniroot()` from stochastic representation, no external dependencies.

Fig.2 Reproduction: Mixture Distribution Power

MMMD vs single-kernel MMD under normal-t mixture



Setup

$$P = (1-p)\mathcal{N}(0,\Sigma_0) + p \cdot t_\nu(0,\Sigma_0)$$
$$Q = (1-p)\mathcal{N}(0,\Sigma_1) + p \cdot t_\nu(0,\Sigma_1)$$
$$\Sigma_1 = 1.25 \cdot \Sigma_0,\ d = 30,\ \rho = 0.5,\ \nu = 10$$

p	MMMD-Mix	MMMD-Gauss	MMMD-LAP	MMD-Gauss
0.0	**0.96**	0.89	0.94	0.46
0.4	0.80	0.84	0.71	0.31
1.0	0.58	0.69	0.53	0.35

MMMD methods: 2–5$\times$ the power of single-kernel baselines

MMMD advantage persists across all mixture proportions p.

Numerical Extensions — Summary

Key findings from the MMMD kernel framework

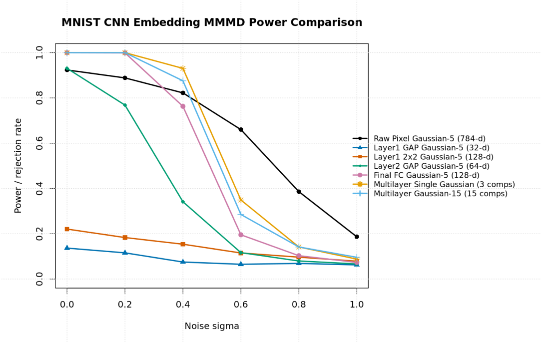
Extension	Main Finding
ε-sensitivity	MMMD detects contamination at $\varepsilon_{\min} = 0.625$; power rises sharply in $[0.45, 0.65]$.
Variance k-sensitivity	MMMD detects $k_{\min} = 1.25$ vs. Gauss MMD $k_{\min} = 1.40$; 15% earlier, 4$\times$ gap at $k = 1.20$.
Type-I & ROC	Well-calibrated at $r = 3, 5$; $r = 10$ GEXP inflates (0.12); LAP robust at all r. ROC confirms test validity.
Graph-MMMD	Type-I ≤ 0.053 across 4 null distributions and 6 skewness levels (up to std. skewness 0.90).
Fig.2 Mixture	MMMD 2–5$\times$ single-kernel power across all p; Mixed MMMD best at $p = 0$ (0.96 vs. 0.46).

All experiments: $d = 30$, AR(1) $\rho = 0.5$, $B = 500$, $N_{\text{trials}} = 150$–200. Modular R/ library with parallel backend.

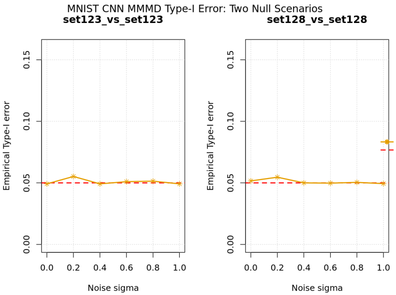
MNIST Reproduction

Original-paper benchmark check

Power



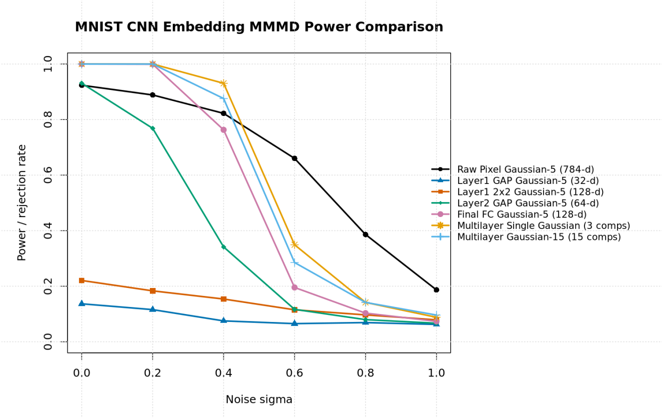
Type-I error



Reproduction anchor: MNIST additive-noise power plus matched-null calibration.

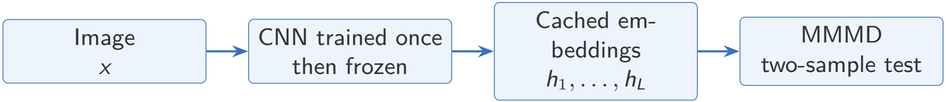
Representation Choice

Raw pixels versus CNN embeddings for image two-sample testing



- Original MMMD improves kernel and bandwidth choice for a fixed input space.
- I compare raw pixels with CNN embeddings.
- Train CNN once, freeze it, cache embeddings, and run MMMD repeatedly.
- Multilayer variants: 3 components with one Gaussian per layer, or 15 components with five bandwidths per layer.

Use MMMD on learned image features, not only on flattened pixels.

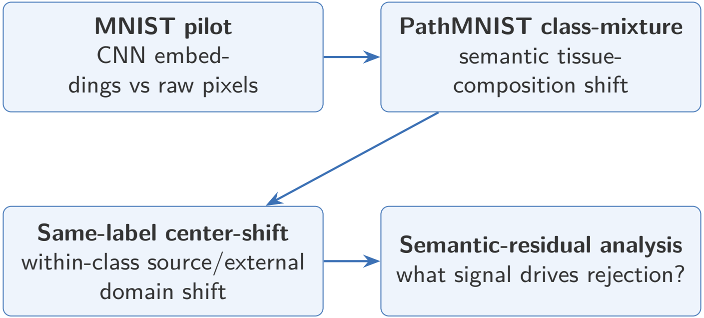


Penultimate FC-128 embedding

- compact, classifier-proximal representation
- enriched for tissue-discriminative information

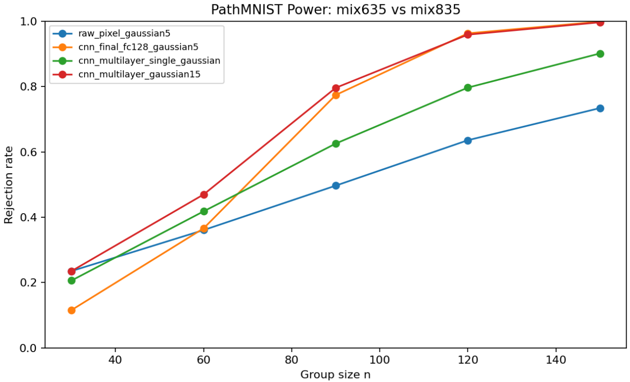
Multilayer embedding

- combines shallow, middle, and penultimate features
- keeps complementary low- and high-level cues
- extends MMMD from multi-kernel to multi-representation aggregation


MNIST pilot
CNN embed-dings vs raw pixels
PathMNIST class-mixture
semantic tissue-composition shift
Same-label center-shift
within-class source/external domain shift
Semantic-residual analysis
what signal drives rejection?

PathMNIST Class-Mixture

Semantic tissue-composition shift



Task

`mix635` vs `mix835`: two shared tissue components, one component changed.

Result

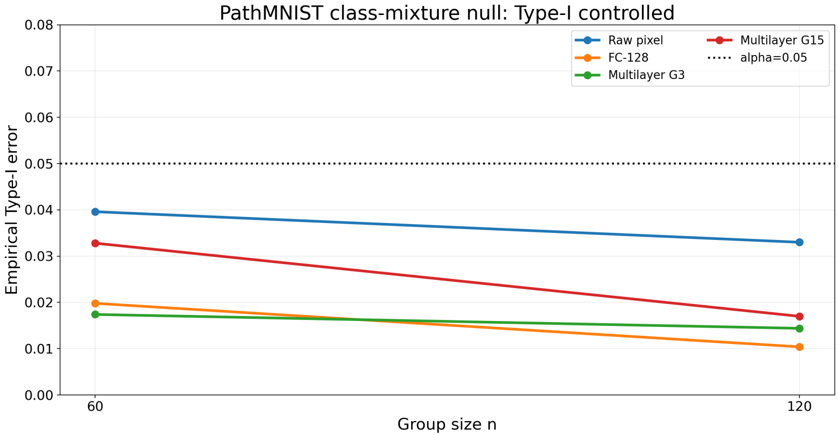
CNN final and multilayer embeddings improve power over raw pixels.

Method	$n = 90$	$n = 120$
Raw pixel	0.497	0.636
FC-128	0.774	**0.963**
Multilayer G15	**0.796**	0.959

Class-Mixture Type-I Check

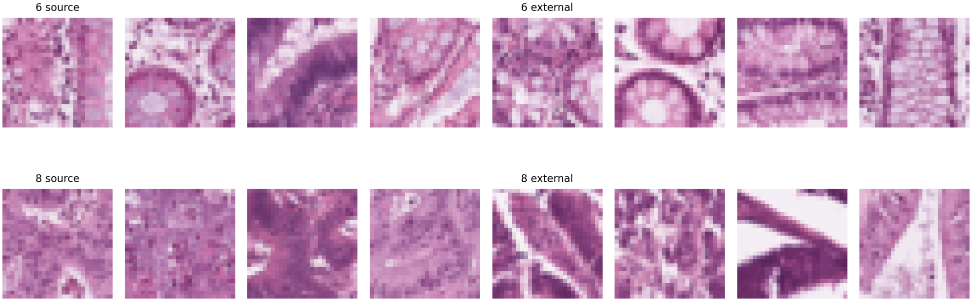
Matched null: `mix635` vs `mix635`

Matched-null rejection stays below $\alpha = 0.05$.



Same-Label Center Shift

A biomedical domain-shift test

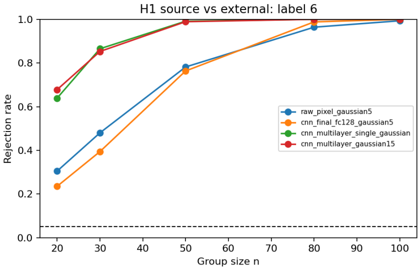


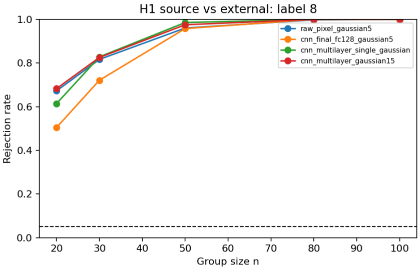
Source / internal: official train holdout.

External: official external test center.

Center-Shift Result

Same-label shift is strong, but representation profiles differ





Rejection rate at $n = 50$

Method	Label 6	Label 8
Raw pixel	0.782	0.960
FC-128	0.763	0.957
Multilayer G15	0.989	0.975
Multilayer single	**0.991**	**0.985**

- Label 6: multilayer is much stronger than raw / FC-128.
- Label 8: raw pixel and FC-128 are already strong.
- Representation profile differs by label; next we try to locate the signal.

Semantic vs Residual Signal

What part of FC-128 drives rejection?

In a same-label test, MMMD rejection may come from different directions of the CNN representation.

$$\text{logits} = Wh + b, \qquad W_c = \left(I - \frac{1}{C}\mathbf{1}\mathbf{1}^T\right)W, \qquad h = h_{\mathrm{sem}} + h_{\mathrm{res}}.$$

Classifier-used part

$$h_{\mathrm{sem}} = P_{\mathrm{row}(W_c)}h$$

- affects class-relative logits
- aligned with tissue-discriminative directions

Classifier-ignored part

$$h_{\mathrm{res}} = h - h_{\mathrm{sem}}$$

- invisible to the linear classifier head
- may still contain domain or stain signal

Diagnostic decomposition: used to interpret the rejection source, not to claim FC-128 is always strongest.

Class-mixture is semantic/logit-aligned; label-8 center shift is residual-dominated.

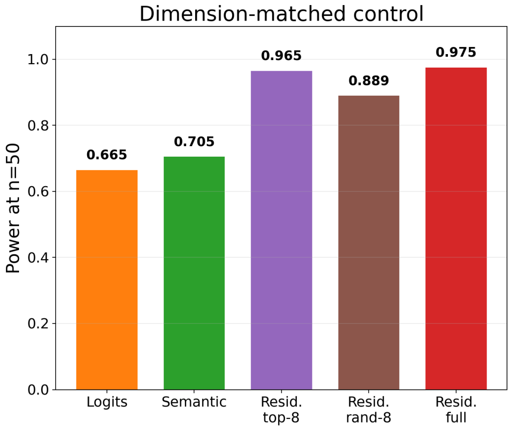
	Logits	Semantic	Residual
Class mix $n = 120$	0.980	0.958	0.955
Label 6 shift $n = 50$	0.858	0.825	0.750
Label 8 shift $n = 50$	0.820	0.840	0.958

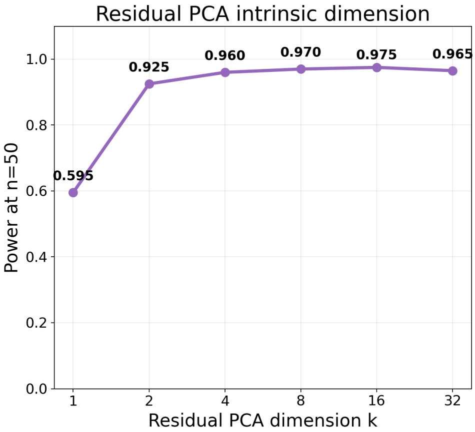
Entries are MMMD rejection rates at representative sample sizes.

Label-8 Residual Checks

Residual signal survives dimension matching and concentrates in few PCs

Label-8 shift is residual-dominated and low-dimensional.

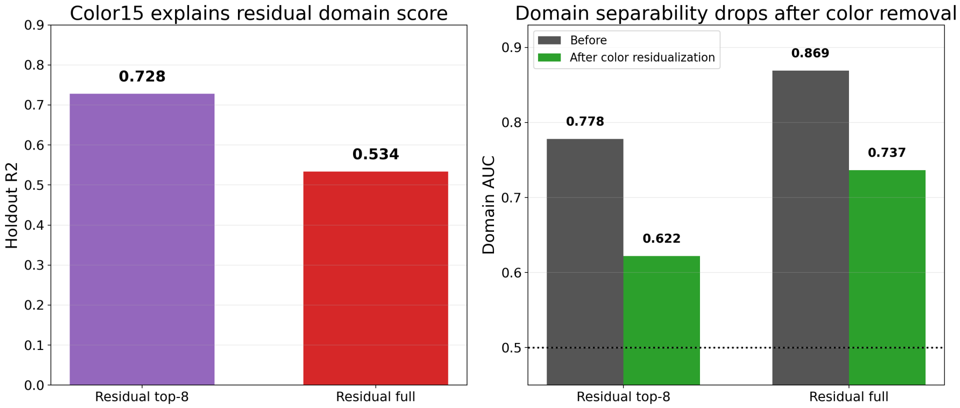




Color/Stain Explanation I

Color15 explains residual domain score

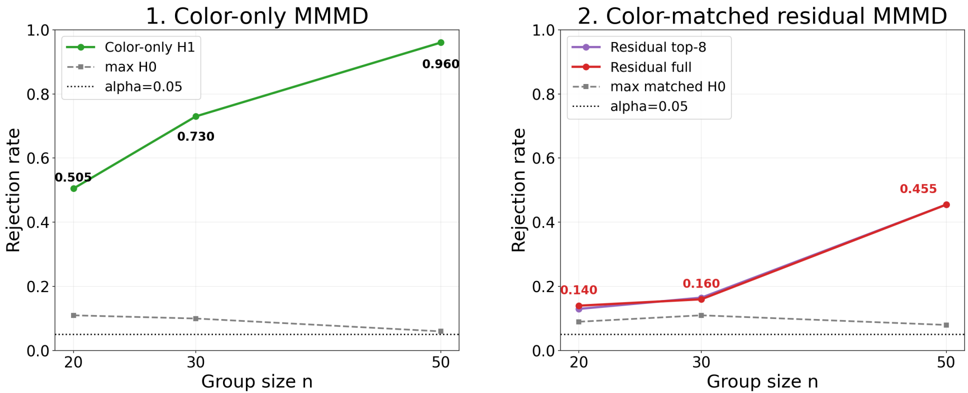
Color/stain explains a large part of the label-8 residual shift.



Color/Stain Explanation II

Color15 → propensity matching → residual MMMD

Color matching weakens residual signal; do not overclaim morphology.



Foundation-Encoder Component

A two-axis extension of the CNN-MMMD experiments

Keep the two-sample test fixed where possible, and vary the image representation explicitly.

Axis 1: image representation

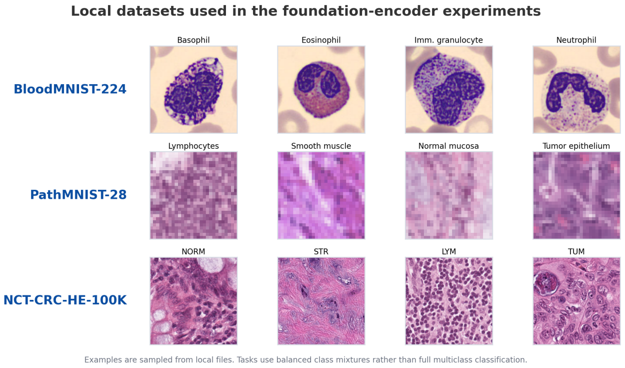
- Raw pixels: direct baseline for image-space MMMD.
- CLIP: frozen image-language foundation encoder.
- DINOv2: frozen self-supervised visual encoder.
- CNN: task-trained feature extractor from the reproduction pipeline.

Axis 2: testing layer

- Original paper: Gaussian multi-kernel MMMD, mainly GAUSS5/GEXP-5.
- lrj extension: NEW-MMMD with covariance stabilization and kernel pruning.
- Metrics: power, Type-I error, and sample-size efficiency.

Datasets Used in the Foundation-Encoder Experiments

Local biomedical image datasets and mixture tasks



BloodMNIST-224
17,092 blood-cell images. Main task: $(0, 1, 3)$ vs $(0, 1, 6)$ under six noise levels.

PathMNIST-28
107,180 pathology patches. Used for the dechao-aligned mixture comparison.

NCT-CRC
100,000 H&E patches. Main task: $(\mathrm{NORM}, \mathrm{STR}, \mathrm{LYM})$ vs $(\mathrm{TUM}, \mathrm{STR}, \mathrm{LYM})$.

Roles in this section

- BloodMNIST: noise robustness.
- PathMNIST: CNN comparison.
- NCT-CRC: sample size and NEW-MMMD.

CLIP and DINOv2

Frozen visual foundation encoders used as image representations

CLIP image encoder

- Trained with image–text contrastive learning on large-scale web data.
- Produces a compact image embedding aligned with language-level semantics.
- Useful as a broad zero-shot representation, but may under-emphasize fine biomedical morphology.
- In this project: images are resized/preprocessed, then encoded once and cached.

DINOv2 visual encoder

- Self-supervised vision transformer trained to learn visual structure without labels.
- Produces morphology-sensitive features that are often strong for pathology-style images.
- Does not use dataset labels during feature extraction, so it is a frozen representation baseline.
- In this project: DINOv2 embeddings are compared with CLIP, raw pixels, and trained CNN features.

Both encoders are used only as feature extractors; the two-sample testing step is still performed by MMMD-style methods.

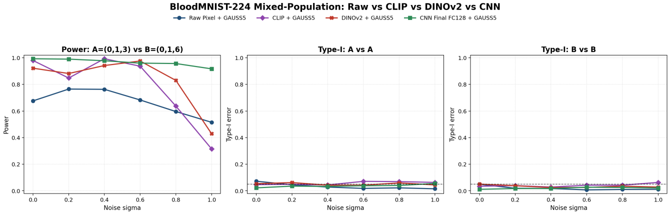
Image datasets BloodMNIST / PathMNIST / NCT-CRC → Feature extraction Raw / CLIP / DINOv2 / CNN → Cached feature pools with labels and noise level → Two-sample testing GAUSS5 or NEW-MMMD

Study	Main comparison	Main question
BloodMNIST noise	Raw / CLIP / DINOv2 / CNN across six noise levels	Which representation is robust to image noise?
BloodMNIST sample size	Raw / CLIP / DINOv2 at $\sigma = 0.6$	How many samples are needed for useful power?
PathMNIST alignment	CLIP / DINOv2 versus trained CNN baselines	Do frozen encoders match the reproduction CNN?
NCT-CRC testing layer	GEXP-5 versus NEW-MMMD across representations	Does stabilization improve power or calibration?

This organization avoids mixing three different factors: dataset difficulty, representation quality, and testing-layer stabilization.

BloodMNIST-224 Noise Study

Task-trained CNN is most robust under image noise



Question: which representation survives additive image noise?

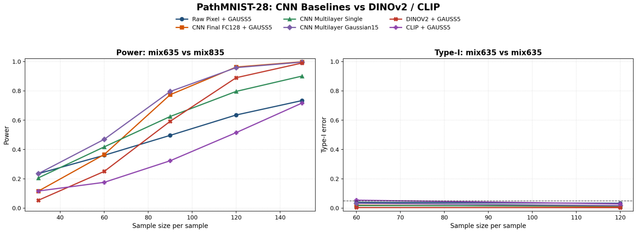
Task: balanced mixtures $(0, 1, 3)$ vs $(0, 1, 6)$.

Method	$\sigma = 0$	$\sigma = 1$
Raw	0.676	0.515
CLIP	0.980	0.315
DINOv2	0.922	0.429
CNN final	**0.993**	**0.917**

- CLIP and DINOv2 are strong at low-to-medium noise.
- CNN final features retain high power even at heavy noise.
- Matched-null Type-I error stays close to nominal after correcting the fixed-pool null design.

PathMNIST-28 Alignment Study

DINOv2 catches up with larger sample size



Question: can frozen encoders match the trained-CNN reproduction baseline?

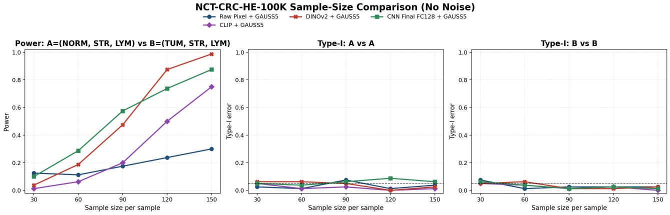
Task: reproduction-aligned overlapping-mixture setting.

n	CLIP	DINOv2
30	0.117	0.053
60	0.176	0.251
90	0.323	0.593
120	0.515	0.890
150	0.717	0.990

- DINOv2 is much stronger than CLIP once sample size grows.
- Small images still favor task-trained CNNs in the small-to-medium sample regime.

NCT-CRC Representation Study

DINOv2 overtakes CNN at larger sample sizes



Question: how do representations scale with sample size on larger pathology images?

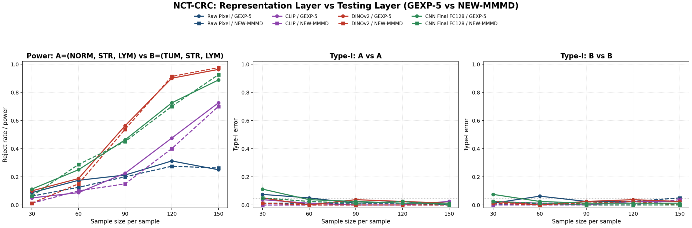
Task: $(\mathrm{NORM}, \mathrm{STR}, \mathrm{LYM})$ vs $(\mathrm{TUM}, \mathrm{STR}, \mathrm{LYM})$.

n	Raw	CLIP	CNN	DINO
30	.125	.013	.100	.038
60	.113	.063	.288	.188
90	.175	.200	.575	.475
120	.238	.500	.738	.875
150	.300	.750	.875	**.988**

- CNN is stronger at small-to-medium sample sizes.
- DINOv2 becomes best when sample size reaches 120–150.
- Raw pixels remain sample-inefficient.

NEW-MMMD Testing-Layer Study

Stabilization helps calibration more than it guarantees power



Question: after fixing the representation, does the stabilized test improve the final decision?

Comparison: GEXP-5 versus NEW-MMMD on NCT-CRC.

- NEW-MMMD uses covariance stabilization and kernel pruning.
- It often lowers Type-I error and improves conditioning.
- It does not uniformly increase power; small samples can lose power.
- At large *n*, it can match GEXP-5 for DINOv2 and CNN.

Takeaways from Foundation-Encoder Experiments

Interpreting the empirical patterns

Two-sample testing is strongly shaped by the geometry of the representation space.

Observation	Interpretation
Raw pixels are weakest	No invariance; high-dimensional distances are noisy and dominated by nuisance variation.
CNN is strongest on BloodMNIST	Task-trained cell morphology features; convolution and pooling filter pixel-level noise.
DINOv2 wins on large NCT-CRC samples	Pretrained visual features capture richer tissue architecture; advantage appears at larger n.
DINOv2 $>$ CLIP	Morphology and texture matter more than language-level semantics.
NEW-MMMD is conservative	Better calibration and numerical stability; possible small-sample power tradeoff.

A good embedding makes biologically meaningful differences kernel-detectable; raw pixels leave MMMD to fight high-dimensional nuisance variation directly.